

Portfolio

Projekt: Data-Mart-Erstellung in SQL

1. 3. Finalisierungsphase

Katharina Bloeck

Einleitung

Im Rahmen dieses Projektes wurde eine relationale Datenbank für eine Buchtausch-App entwickelt und mit dem Datenbankmanagementsystem SQLite umgesetzt. Ziel der Datenbank ist die strukturierte Speicherung und Verwaltung der Informationen, die für den Ausleihprozess von Büchern erforderlich sind. Hierzu zählen unter anderem Benutzer:innen, Rollen, Bücher, Buchexemplare, Standorte, Angebote, Ausleihtransaktionen und Bewertungen.

Die Datenbank wurde auf Grundlage des zuvor entwickelten Entity-Relationship-Modells implementiert und anschließend um Testdaten, Integritätsregeln und Optimierungen ergänzt. Primär- und Fremdschlüssel gewährleisten die Beziehungen zwischen den Tabellen, während Constraints die Konsistenz der gespeicherten Daten sicherstellen. Zusätzlich wurden Indizes auf häufig verwendete Fremdschlüsselattribute angelegt, um die Performance von Join- und Such-Abfragen zu verbessern.

Im Folgenden werden die wichtigsten Funktionen des Datenbankmanagementsystems sowie ausgewählte Metadaten und technische Eigenschaften der Implementierung vorgestellt.

Metadaten

Datenbankmanagementsystem: SQLite (Version 3.46.1)

Anzahl der Tabellen: 15

Größe der Datenbank: 94 KB

Anzahl der Datensätze pro Tabelle:

Tabelle	Anzahl Datensätze
user	11
role	2
userRole	13
author	12
publisher	10
language	10
genre	10
book	10
bookAuthor	12
bookGenre	16
copy	13
location	14
offer	14
transactions	11
review	12

Tabelle 1 - Anzahl der Datensätze je Tabelle

Funktionalität des Datenbankmanagementsystems

Die entwickelte Datenbank unterstützt die wesentlichen Prozesse einer Buchtausch-App und ermöglicht die strukturierte Verwaltung aller relevanten Informationen. Durch die Verknüpfung der einzelnen Tabellen können sowohl Stammdaten als auch Ausleihvorgänge konsistent gespeichert und ausgewertet werden.

Benutzer- und Rollenverwaltung

Die Datenbank verwaltet registrierte Benutzer sowie deren Rollen innerhalb der Anwendung. Über eine Zwischentabelle können Benutzer mehrere Rollen gleichzeitig benutzen und beispielsweise sowohl Bücher anbieten als auch ausleihen.

Verwaltung von Büchern und Buchexemplaren

Die Stammdaten eines Buches werden getrennt von den einzelnen Buchexemplaren gespeichert. Dadurch können mehrere Benutzer:innen dasselbe Buch besitzen, ohne dass redundante Informationen gespeichert werden müssen. Zusätzlich werden Autor, Verlage, Sprachen und Genres in separaten Tabellen verwaltet und über Beziehungen mit den Büchern verknüpft.

Verwaltung von Angeboten und Standorten

Für jedes Buchexemplar können Angebote mit individuellen Leihbedingungen angelegt werden. Hierzu zählen unter anderem die maximale Leihdauer, die Versandoption sowie der zugehörige Standort. Die Speicherung verschiedener Standorttypen ermöglicht sowohl persönliche Übergaben an einem Treffpunkt als auch Abholungen des Buches bei der anbietenden Person sowie den Versand.

Verwaltung von Ausleihvorgängen

Ausleihanfragen und Ausleihvorgänge werden in einer eigenen Transaktionstabelle dokumentiert. Dabei werden unter anderem die ausleihende Person, das ausgewählte Angebot sowie der aktuelle Bearbeitungsstatus gespeichert. Dadurch können sämtliche Ausleihvorgänge nachvollzogen und verwaltet werden.

Bewertungen

Nach dem Lesen eines Buches können Benutzer:innen Bewertungen und Kommentare hinterlassen. Die Bewertungen beziehen sich auf das jeweilige Buch und ermöglichen eine Einschätzung für andere Benutzer:innen.

Beispielabfrage

```
SELECT
  b.title AS Buch,
  u.first_name || ' ' || u.last_name AS Ausleiher,
  t.status AS Status,
  DATE(t.requested_at) AS Anfrage,
  DATE(t.due_at) AS Rückgabe_bis
FROM transactions t
JOIN user u
  ON t.borrower_user_id = u.user_id
JOIN offer o
  ON t.offer_id = o.offer_id
JOIN copy c
  ON o.copy_id = c.copy_id
JOIN book b
  ON c.book_id = b.book_id
ORDER BY t.requested_at DESC;
```

Abbildung 1 - SQL-Abfrage

	Buch	Ausleiher	Status	Anfrage	Rückgabe_bis
1	Der Hobbit	Lisa Fischer	REQUESTED	2026-04-25	NULL
2	Il Codice Da Vinci	Eva Musterfrau	APPROVED	2026-04-22	2026-05-06
3	Harry Potter und der Stein der Weisen	Lisa Fischer	REQUESTED	2026-04-20	NULL
4	1984	Jonas Becker	ACTIVE	2026-04-18	2026-05-02
5	Mord im Orientexpress	Tom Meyer	APPROVED	2026-04-15	2026-04-29
6	The Shining	Max Mustermann	ACTIVE	2026-04-10	2026-04-24
7	1984	Laura Wagner	CANCELLED	2026-04-05	NULL
8	The Shining	Eva Musterfrau	RETURNED	2026-04-01	2026-04-15
9	The Shining	Eva Musterfrau	RETURNED	2026-04-01	2026-04-15
10	Harry Potter und der Stein der Weisen	Anna Schmidt	RETURNED	2026-03-10	2026-03-20
11	Foundation	Lukas Schulz	RETURNED	2026-03-01	2026-03-15

Abbildung 2 - Ergebnis der SQL-Abfrage

Technische Bewertung und Optimierung

Normalisierung

Die Datenbank wurde relational und normalisiert aufgebaut, um Redundanzen zu reduzieren und Inkonsistenzen zu vermeiden. Die erste Normalform wird eingehalten, da die Attribute atomar gespeichert werden und keine Mehrfachwerte innerhalb einzelner Tabellenfelder vorkommen. Die zweite Normalform wird durch die eindeutige Abhängigkeit der Nichtschlüsselattribute vom jeweiligen Primärschlüssel sichergestellt. Die dritte Normalform wird insbesondere durch die Auslagerung wiederverwendbarer Stammdaten wie Autor:innen, Verlage, Sprachen und Genres in eigene Tabellen sowie durch die Umsetzung von n:m-Beziehungen über Zwischentabellen erreicht.

Datenintegrität

Zur Sicherstellung der Datenintegrität wurden Primärschlüssel, Fremdschlüssel und Constraints verwendet. Primärschlüssel identifizieren Datensätze eindeutig, während Fremdschlüssel die referentielle Integrität zwischen den Tabellen sichern. Zusätzlich verhindern NOT NULL-, UNIQUE- und CHECK-Constraints ungültige oder widersprüchliche Werte, beispielsweise bei Statuswerten, Bewertungen oder Zustandsangaben von Buchexemplaren.

Performance

Zur Verbesserung der Abfrageleistung wurden Indizes auf häufig verwendete Fremdschlüsselspalten angelegt. Diese unterstützen insbesondere JOIN-Abfragen zwischen zentralen Tabellen wie Buchexemplaren, Angeboten, Ausleihtransaktionen und Bewertungen. Dadurch können typische Abfragen, beispielsweise zur Anzeige von Angeboten oder Ausleihvorgängen, effizienter ausgeführt werden.

Mögliche Erweiterungen

Einige fachliche Regeln könnten in einer zukünftigen Version zusätzlich auf Datenbankebene abgesichert werden. Dazu zählen beispielsweise die Verhinderung der Ausleihe eigener Bücher, die Einschränkungen von Bewertungen auf abgeschlossene Ausleihvorgänge oder die Sperrung paralleler aktiver Ausleihen desselben Exemplars. Solche Regeln lassen sich mit einfachen Constraints nicht vollständig umsetzen, da hierfür Informationen aus mehreren Tabellen berücksichtigt werden müssen. Eine mögliche Lösung wäre der Einsatz von Triggern, welche entsprechende Prüfungen vor dem Einfügen oder Ändern von Datensätzen automatisch durchführen. Alternativ können diese Regeln auch in der Anwendungsschicht der Buchtausch-App umgesetzt werden.

Link zum GitHub-Repository

<https://github.com/kabl-dot/Buchtauschapp>